

Дизайн и анализ на алгоритми — консултация

Станислав Игов (Стъката)

2025/2026

Консултация за изпит по ДАА

Задача 1: Лодки за спасяване на хора (Boats to Save Kids)

Условие: На острова Little Saint James са попаднали n деца (Не е ясно как и поради какви обстоятелства). Тези деца трябва да се спасят. На острова има неограничен запас от лодки. Трябва да проверим обаче с колко най-малко лодки можем да спасим всички деца при следните условия. Даден е масив $K[1..n]$, където $K[i]$ е теглото на i -тото дете и число limit — максималното тегло, което една лодка може да носи.

Ограничение: Всяка лодка може да носи **най-много две деца** едновременно, при условие че сумата от теглата им не надвишава limit .

Изход: Минималният брой лодки, необходими за да се превозят всички деца.

Допускане: $K[i] \leq \text{limit}$ за всяко i (иначе детето не може да се качи на никоя лодка и задачата няма решение).

Мотивация

Лодка побира най-много две деца, така че всъщност *сдвояваме* децата (и евентуално оставяме някои сами), минимизирайки броя групи. Естественят въпрос е: *с кого да сдвоим най-тежкото дете?*

Ключово наблюдение: Разгледай най-тежкото дете H . То трябва да отиде на някоя лодка. Има само два варианта — сам или с още един. Ако *може* да сподели лодка с някого, то най-разумният съпътник е **най-лекият** наличен човек L :

- Ако дори най-лекият не се събира с H (т.е. $L + H > \text{limit}$), то *никой* не се събира с H (всички са поне толкова тежки като L) — значи H задължително пътува сам.
- Ако $L + H \leq \text{limit}$, сдвояваме ги. Така „изхбяваме“ най-тежкия (който е най-труден за настаняване) заедно с най-лекия (който най-лесно се сдвоява с който и да е друг по-късно). Това е безопасен избор — ще го докажем с exchange argument.

Това подсказва: **сортирай** хората по тегло и работи с **два указателя** — един към най-лекия, един към най-тежкия.

Алгоритъм

```
BoatsToSaveKids(K[1..n], limit):
@1  mergeSort(K)                                // O(n log n)
@2  i <- 1; j <- n; boats <- 0
@3  while i <= j do
@4      if K[i] + K[j] <= limit:
@5          i <- i + 1                            // най-лекият също се качва при най-тежкия
@6      endif
@7      j <- j - 1                                // най-тежкият винаги се качва на тази лодка
@8      boats <- boats + 1
@9  endwhile
@10 return boats
```

Забележка за последното дете: когато $i = j$ (остава едно дете), условието проверява $2 \cdot [i]$; независимо дали е $\leq \text{limit}$ или не, изпълнява се $j \leftarrow j - 1$ и `boats++` точно веднъж, т.е. този човек получава своя (самостоятелна) лодка. Коректно.

Дали алгоритъма завършва: Да забележим, че ред @7 винаги се изпълнява като след всяко изпълнение j намаля с 1. В някакъв момент със сигурност $j = i$, понеже $i < j$.

Сложност: сортирането е $O(n \log n)$; цикълът с двата указателя прави $O(n)$ стъпки (всяка стъпка придвижва поне j). **Общо:** $O(n \log n)$ по време, $O(1)$ допълнителна памет (извън сортирането).

Коректност: оптималност чрез exchange argument

Нека сортираните тегла са $a_1 \leq a_2 \leq \dots \leq a_n$. Алчният избор за текущата (под)задача е:

ако $a_1 + a_n \leq \text{limit}$ — сложи най-лекия a_1 и най-тежкия a_n в една лодка; иначе — най-тежкия a_n пътува сам.

Лема (безопасност на алчния избор). Съществува оптимално решение, което прави същия пръв избор като алчния алгоритъм.

Доказателство. Разглеждаме двата случая.

Случай А ($a_1 + a_n > \text{limit}$). Тъй като a_1 е най-лекият, за всеки човек a_k имаме $a_k + a_n \geq a_1 + a_n > \text{limit}$. Значи a_n не може да сподели лодка с никого, т.е. във всяко допустимо решение a_n пътува сам. В частност има оптимално решение, в което a_n е сам — точно изборът на алчния. ✓

Случай В ($a_1 + a_n \leq \text{limit}$). Нека OPT е произволно оптимално решение. Преобразуваме го в оптимално решение, в което a_1 и a_n са заедно, без да увеличаваме броя лодки.

Нека в OPT човекът a_n е на лодка X , а a_1 — на лодка Y .

- Ако $X = Y$: a_1 и a_n вече са заедно — готово.
- Ако $X \neq Y$: Нека p е (евентуалният) съпътник на a_n в X , а q — (евентуалният) съпътник на a_1 в Y . Прегрупираме: слагаме $\{a_1, a_n\}$ в една лодка, а $\{p, q\}$ (тези, които съществуват) — в друга. Останалите лодки не пипаме.

Това прегрупиране е **допустимо**:

- $a_1 + a_n \leq \text{limit}$ по предположението на Случай В.
- Ако и p , и q съществуват: p е бил съпътник на a_n , значи $p + a_n \leq \text{limit}$. Понеже a_n е най-тежкия, $q \leq a_n$, откъдето

$$p + q \leq p + a_n \leq \text{limit}.$$

Тоест $\{p, q\}$ се събират в една лодка. (Ако само един от p, q съществува — той пътува сам, тривиално допустимо; ако никой — няма втора лодка.)

Броят лодки **не нараства**: лодките X и Y се заменят с лодката $\{a_1, a_n\}$ и (най-много) още една лодка $\{p, q\}$ — т.е. най-много толкова лодки, колкото преди. Следователно полученото решение е също оптимално и в него a_1, a_n са заедно. ✓

(Случаят „и a_1 , и a_n са сами на отделни лодки в OPT “ не може да възникне при оптимално OPT : бихме ги слели в една лодка и намалили броя лодки — противоречие. Иначе казано, той се покрива от горния анализ при липсващи p и q .) Лемата е доказана. ■

Теорема. Алчният алгоритъм използва минималния брой лодки.

Доказателство (индукция по броя хора n).

База: $n = 0$ — 0 лодки; $n = 1$ — 1 лодка. И двете очевидно оптимални.

Индукционна стъпка: Допускаме, че алгоритъмът е оптимален за всички инстанции с по-малко от n хора. Нека OPT е оптималният брой лодки за текущата инстанция.

- **Случай А** ($a_1 + a_n > \text{limit}$): Алчният слага a_n сам и продължава с $\{a_1, \dots, a_{n-1}\}$. По лемата a_n е сам във всяко решение, затова $OPT = 1 + OPT'$, където OPT' е оптимумът за останалите $n - 1$ хора. По индукционна хипотеза алгоритъмът решава остатъка с OPT' лодки. Общо: $1 + OPT' = OPT$. ✓

- **Случай В** ($a_1 + a_n \leq \text{limit}$): Алчният вдвоява a_1, a_n и продължава с $\{a_2, \dots, a_{n-1}\}$. По лемата има оптимално решение, в което a_1, a_n са заедно; махайки тяхната лодка, остава допустимо решение за останалите хора с $OPT - 1$ лодки, и то е оптимално за остатъка (иначе, добавяйки обратно лодката $\{a_1, a_n\}$, бихме подобрили OPT — противоречие). Значи $OPT' = OPT - 1$. По индукционна хипотеза алгоритъмът решава остатъка с OPT' лодки. Общо: $1 + OPT' = 1 + (OPT - 1) = OPT$. ✓

И в двата случая алгоритъмът използва точно OPT лодки. ■

Пример

Нека $a = [3, 5, 3, 4]$ и $\text{limit} = 5$. След сортиране: $[3, 3, 4, 5]$, индекси 1..4.

стъпка	i	j	$a_i + a_j$	действие	лодки
1	1	4	$3 + 5 = 8 > 5$	$a_4 = 5$ сам; $j \leftarrow 3$	1
2	1	3	$3 + 4 = 7 > 5$	$a_3 = 4$ сам; $j \leftarrow 2$	2
3	1	2	$3 + 3 = 6 > 5$	$a_2 = 3$ сам; $j \leftarrow 1$	3
4	1	1	$3 + 3 = 6 > 5$	$a_1 = 3$ сам; $j \leftarrow 0$	4

Отговор: 4 лодки (никои двама не се събират заедно при $\text{limit} = 5$).

Втори пример: $a = [3, 2, 2, 1]$, $\text{limit} = 3$. Сортирано: $[1, 2, 2, 3]$.

стъпка	i	j	$a_i + a_j$	действие	лодки
1	1	4	$1 + 3 = 4 > 3$	$a_4 = 3$ сам; $j \leftarrow 3$	1
2	1	3	$1 + 2 = 3 \leq 3$	сдвояваме 1, 2; $i \leftarrow 2, j \leftarrow 2$	2
3	2	2	$2 + 2 = 4 > 3$	$a_2 = 2$ сам; $j \leftarrow 1$	3

Отговор: 3 лодки — например $\{3\}$, $\{1, 2\}$, $\{2\}$. По-малко от 3 е невъзможно: четирима души, лодка побира най-много двама, значи нужни са поне $\lceil 4/2 \rceil = 2$; но човекът с тегло 3 запълва лодка сам (с всеки друг надхвърля limit), което принуждава поне 3.

Задача 2: Bangaranga

Условие: Виена е представена като граф $G = (V, E)$ с $n = |V|$ кръстовища (върхове) и $m = |E|$ улици (ребра); всяка улица свързва точно две кръстовища. За преминаване между две съседни кръстовища по улица e Дара има два избора:

- **пеша** — за време $w(e)$;
- **с такси** — за време $t(e)$.

Уловка: при всяко *качване* в такси Дара изчаква 1 единица време, докато таксито пристигне. Това чакане се плаща **еднократно** при качване; докато е *вътре* в таксито, тя *не* чака на следващите кръстовища. Слизането от таксито е безплатно (0 единици време). Дара може многократно да слиза и да се качва.

Вход: Граф $G = (V, E)$ с $|V| = n$, $|E| = m$, и две теглови функции $w : E \rightarrow \mathbb{Q}^+$ и $t : E \rightarrow \mathbb{Q}^+$. Стартов връх 1 (хотелът) и целеви връх n (залата).

Изход: Минималното време за придвижване от връх 1 до връх n .

Мотивация

На пръв поглед това е задача за най-къс път: всяко ребро има „цена“ и търсим най-евтиния маршрут от 1 до n . Но цената на едно преминаване с такси *не е фиксирана*: ако Дара *вече* е в такси, реброто струва само $t(e)$; ако *сега се качва*, струва $1 + t(e)$ заради чакането. Иначе казано, **цената на едно ребро зависи от това в какво „състояние“ се намира Дара**, когато го преминава.

Ключово наблюдение: Скритото състояние има точно две стойности — *пеша* или *в такси*. Това е класически сигнал за **разширяване на графа**: правим по едно копие на всеки връх за всяко състояние, кодираме преходите (качване, слизание, ходене, каране) като ребра, и свеждаме задачата до обикновен най-къс път в по-голям граф, върху който пускаме Dijkstra.

Какво прави алгоритъмът на Dijkstra (използван „наготово”)

Какъв проблем решава: Дадени са насочен граф $G' = (V', E')$ с неотрицателна теглова функция $\ell : E' \rightarrow \mathbb{Q}_{\geq 0}$ и стартов връх s . Dijkstra намира дължината на най-късия път от s до всеки връх.

Какво връща: Масив $\text{dist}[\cdot]$, такъв че $\text{dist}[x]$ е дължината на най-късия път от s до x (или $+\infty$, ако x е недостижим).

Как работи: Поддържа приоритетна опашка по минимум с текущи оценки за разстоянията. Многократно изважда непосетен връх u с минимална текуща оценка, финализира го и *релаксира* изходящите му ребра (за всяко (u, x) проверява дали $\text{dist}[u] + \ell(u, x) < \text{dist}[x]$ и ако да — обновява). Инвариантът е, че при първото изваждане на връх неговото dist вече е окончателно — което важи **само** при $\ell \geq 0$. При имплементация с бинарна купа сложността е $O((|V'| + |E'|) \log |V'|)$.

Стъпка 1: Конструкция на разширения граф G'

Изграждаме нов **насочен** граф $G' = (V', E')$ с по две копия на всеки връх:

$$V' = \{v_F : v \in V\} \cup \{v_T : v \in V\},$$

където v_F означава „Дара е на връх v , **пеша**”, а v_T — „Дара е на връх v , **в такси**”. Така $|V'| = 2n$.

Реброто на G' кодират четирите възможни елементарни действия:

1. **Ходене** (изисква да е пеша). За всяка улица $\{u, v\} \in E$ добавяме

$$u_F \rightarrow v_F \quad \text{и} \quad v_F \rightarrow u_F, \quad \text{с тегло } w(\{u, v\}).$$

2. **Каране** (изисква вече да е в такси — без чакане). За всяка улица $\{u, v\} \in E$ добавяме

$$u_T \rightarrow v_T \quad \text{и} \quad v_T \rightarrow u_T, \quad \text{с тегло } t(\{u, v\}).$$

3. **Качване в такси** (плаща се чакането 1). За всеки връх $v \in V$ добавяме

$$v_F \rightarrow v_T, \quad \text{с тегло } 1.$$

4. **Слизане от такси** (безплатно). За всеки връх $v \in V$ добавяме

$$v_T \rightarrow v_F, \quad \text{с тегло } 0.$$

Защо точно тези ребра? Чакането 1 се плаща *само* на реброто за качване $v_F \rightarrow v_T$ — т.е. **веднъж** на качване. Веднъж в слой „такси”, последователни кариния $u_T \rightarrow v_T \rightarrow x_T$ струват само t всяко, без допълнително чакане — точно както изисква условието („докато е вътре, не чака на следващите кръстовища”). Слизането $v_T \rightarrow v_F$ е с тегло 0 и позволява смяна на режима по всяко време; а понеже след слизание може отново да се качим (плащайки пак 1), многократното качване/слизание е автоматично разрешено.

Брой ребра: $2m$ (ходене, двете посоки) + $2m$ (каране) + n (качване) + n (слизание) = $4m + 2n = O(n + m)$.

Сложност на стъпката: обхождаме всяко ребро и всеки връх по веднъж — $O(n + m)$ време и памет.

Стъпка 2: Пускане на Dijkstra от 1_F

Стартираме Dijkstra върху G' от върха

$$s = 1_F \quad (\text{Дара е в хотела, на връх } 1, \text{ пеша; начален момент } 0).$$

Получаваме масив dist , в който за всяко състояние $x \in V'$ стойността $\text{dist}[x]$ е минималното време, за което Дара може да достигне това състояние.

Защо Dijkstra е приложим? Всички тегла в G' са неотрицателни: $w, t \in \mathbb{Q}^+ > 0$, теглото за качване е $1 > 0$, за слизание — 0. Следователно инвариантът на Dijkstra е валиден и резултатът е коректен.

Сложност на стъпката: G' има $|V'| = 2n$ върха и $|E'| = O(n + m)$ ребра. С бинарна купа:

$$O((|V'| + |E'|) \log |V'|) = O((n + m) \log(2n)) = O((n + m) \log n).$$

Стъпка 3: Извличане на отговора

Дара е стигнала залата, ако е на връх n — независимо в какъв режим. Затова отговорът е

$$T = \min(\text{dist}[n_F], \text{dist}[n_T]).$$

Забележка: понеже съществува безплатно ребро $n_T \rightarrow n_F$ (тегло 0), винаги имаме $\text{dist}[n_F] \leq \text{dist}[n_T]$, тоест на практика $T = \text{dist}[n_F]$. Все пак минимумът от двете е напълно коректен и безопасен начин за записване.

Сложност на стъпката: $O(1)$.

```
Bangaranga(G = (V, E), w, t, n):
@1   build G' = (V', E'):                // O(n + m)
@2   for each street {u, v} in E:
@3       add u_F -> v_F and v_F -> u_F with weight w({u,v}) // ходене
@4       add u_T -> v_T and v_T -> u_T with weight t({u,v}) // каране
@5   for each vertex v in V:
@6       add v_F -> v_T with weight 1 // качване
@7       add v_T -> v_F with weight 0 // слизване
@8   dist <- Dijkstra(G', source = 1_F)    // O((n + m) log n)
@9   return min(dist[n_F], dist[n_T])      // O(1)
```

Защо тези стъпки решават задачата (коректност)

Твърдение: За всеки връх v и режим $\mu \in \{F, T\}$, стойността $\text{dist}[v_\mu]$ е минималното време, за което Дара може да бъде на кръстовище v в режим μ . Следователно $\min(\text{dist}[n_F], \text{dist}[n_T])$ е минималното време да достигне залата.

Идея на доказателството (биекция между маршрути и пътища в G'). Всяко допустимо придвижване на Дара е поредица от елементарни действия — ходене, качване, каране, слизване. Всяко такова действие отговаря точно на едно ребро от G' със *същата* цена:

ходене по $\{u, v\}$	$u_F \rightarrow v_F$	цена $w(\{u, v\})$
качване в такси на u	$u_F \rightarrow u_T$	цена 1
каране по $\{u, v\}$	$u_T \rightarrow v_T$	цена $t(\{u, v\})$
слизване на v	$v_T \rightarrow v_F$	цена 0

Затова всеки маршрут на Дара, започващ от хотела пеша, съответства на път в G' от 1_F със *същото* общо време, и обратно — всеки път в G' от 1_F кодира допустим маршрут. Минималното време за достигане на дадено състояние е следователно дължината на най-късия път до съответния връх в G' , която Dijkstra пресмята коректно (всички тегла ≥ 0).

Накрая, Дара е в залата точно когато е на връх n (в който и да е режим), затова търсеният минимум е $\min(\text{dist}[n_F], \text{dist}[n_T])$. ✓

Обща времева сложност

- Изграждане на G' : $O(n + m)$.
- Dijkstra върху G' ($2n$ върха, $O(n + m)$ ребра, бинарна купа): $O((n + m) \log n)$.
- Извличане на отговора: $O(1)$.

Общо: $O((n + m) \log n)$ по време, $O(n + m)$ по памет. ✓ (отговаря на изискваната сложност)

Пример

Нека $n = 4$ и улиците образуват пътя $1 - 2 - 3 - 4$ със следните тегла:

$$\{1, 2\} : w = 2, t = 5 \quad \{2, 3\} : w = 10, t = 1 \quad \{3, 4\} : w = 2, t = 5.$$

Само пеша: $2 + 10 + 2 = 14$. Изцяло с такси: $1 + 5 + 1 + 5 = 12$.

Оптимално: ходим $1 \rightarrow 2$ (2), качваме се на 2 (+1), караме $2 \rightarrow 3$ (+1), слизаме на 3 (+0), ходим $3 \rightarrow 4$ (+2) — общо 6. Т.е. изгодно е таксито само за евтиното ребро $\{2, 3\}$.

Разширеният граф има 8 върха. Изпълнение на Dijkstra от $1_F = 0$ дава:

състояние	1_F	1_T	2_F	2_T	3_F	3_T	4_F	4_T
dist	0	1	2	3	4	4	6	7

Кратко проследяване на ключовите релаксации:

- От $1_F = 0$: $2_F \leftarrow 2$ (ходене), $1_T \leftarrow 1$ (качване).
- От $2_F = 2$: $2_T \leftarrow 3$ (качване, $2 + 1$) — по-добро от 2_T през 1_T ($1 + 5 = 6$).
- От $2_T = 3$: $3_T \leftarrow 4$ (каране, $3 + 1$).
- От $3_T = 4$: $3_F \leftarrow 4$ (слизване, $4 + 0$).
- От $3_F = 4$: $4_F \leftarrow 6$ (ходене, $4 + 2$).

Отговор: $T = \min(\text{dist}[4_F], \text{dist}[4_T]) = \min(6, 7) = 6$.

Това е причината обикновен най-къс път върху оригиналния граф (без състояния) да не работи: цената на ребро зависи от това дали Дара вече кара такси.

Задача 3: Изкачване на стълба (Stairs)

Условие: Имаме стълба с n стъпала. От всяко стъпало можем да се качим с 1, 2 или 3 стъпала нагоре наведнъж. Започваме от земята (стъпало 0) и искаме да достигнем стъпало n .

Вход: Естествено число n .

Изход: Броят на различните начини, по които можем да стигнем до стъпало n .

Мотивация

Искаме **броя** на начините, а не самите начини. Наивно изброяване е експоненциално (броят на начините расте като последователност от тип трибоначи). Но за *броене* е достатъчно ДП — комбинираме броевете на по-малки подзадачи, без да изброяваме поотделно.

Ключово наблюдение (кое е последното решение?): Всеки начин да стигнем стъпало i завършва с *последна крачка* с размер 1, 2 или 3. Ако последната крачка е s , то непосредствено преди нея сме били на стъпало $i - s$. Тези три случая са **взаимно изключващи се** (размерът на последната крачка е еднозначен) и **изчерпателни** (всеки начин има последна крачка). Значи броят за i е сборът от броевете за $i - 1$, $i - 2$ и $i - 3$.

Стъпка 1: Дефиниция на подзадачата

Нека:

$$dp[i] = \text{броят на различните начини да стигнем от стъпало 0 до стъпало } i.$$

Стъпка 2: Рекурентна връзка

Класифицираме всеки начин за стигане до i по размера на последната крачка:

- последна крачка 1 (от $i - 1$): дава $dp[i - 1]$ начина;
- последна крачка 2 (от $i - 2$): дава $dp[i - 2]$ начина;
- последна крачка 3 (от $i - 3$): дава $dp[i - 3]$ начина.

Сумирайки:

$$dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3]$$

с конвенцията $dp[j] = 0$ за $j < 0$ (не можем да стигнем „отрицателно“ стъпало).

Стъпка 3: Базов случай

$$dp[0] = 1$$

Има точно **един** начин да сме на стъпало 0 — да не правим нито една крачка (празната последователност). Заедно с конвенцията $dp[j] = 0$ за $j < 0$ това определя напълно рекурсията; в частност:

$$dp[1] = dp[0] = 1, \quad dp[2] = dp[1] + dp[0] = 2, \quad dp[3] = dp[2] + dp[1] + dp[0] = 4.$$

Стъпка 4: Ред на пресмятане и псевдокод

$dp[i]$ зависи само от $dp[i-1]$, $dp[i-2]$, $dp[i-3]$ — т.е. от строго по-малки индекси. Затова попълваме таблицата отляво надясно: $i = 1, 2, \dots, n$.

CountStairs(n):

```
@1  if n = 0: return 1
@2  dp[0..n] <- malloc(n+1){0}
@3  dp[0] <- 1
@4  for i from 1 to n:
@5      ways <- dp[i-1]                // последна крачка 1
@6      if i >= 2: ways <- ways + dp[i-2] // последна крачка 2
@7      if i >= 3: ways <- ways + dp[i-3] // последна крачка 3
@8      dp[i] <- ways
@9  endfor
@10 return dp[n]
```

Отговор: $dp[n]$.

Сложност: $O(n)$ по време (един цикъл с константна работа на итерация), $O(n)$ по памет. (Лесно се сваля до $O(1)$ памет, защото всяка стъпка използва само последните три стойности $dp[i-1]$, $dp[i-2]$, $dp[i-3]$.)

Защо рекурентната връзка е коректна?

Двата градивни принципа са, че трите случая (последна крачка 1, 2 или 3) са **изчерпателни** (всеки начин завършва с някоя от тях) и **взаимно изключващи се** (размерът на последната крачка е точно един), затова броят е *сборът*, а не максимум. Освен това подзадачите $i-1$, $i-2$, $i-3$ са *строго по-малки* от i , което гарантира липсата на циклични зависимости и коректен ред на пресмятане.

Коректност (индукция по i)

Твърдение: $dp[i]$ е броят на различните начини да се стигне от стъпало 0 до стъпало i с крачки от $\{1, 2, 3\}$.

База ($i = 0$): Единственият начин да сме на стъпало 0 е празната последователност от крачки — 1 начин. $dp[0] = 1$. ✓

(За $i < 0$: няма допустим начин, 0 начина — конвенция.)

Индукционна стъпка ($i \geq 1$): Предполагаме, че $dp[j]$ е коректно за всички $0 \leq j < i$. Нека $W(i)$ е множеството от всички начини да се стигне до i . Разделяме $W(i)$ по размера s на *последната* крачка:

$$W(i) = W_1(i) \dot{\cup} W_2(i) \dot{\cup} W_3(i),$$

където $W_s(i)$ са начините, чиято последна крачка е с размер s (а предпоследното положение е стъпало $i-s \geq 0$). Това е **разбиване** (дизюнктно и пълно), понеже всеки начин има точно една последна крачка с точно един размер.

За фиксирано s съответствието „премахни последната крачка“ е **биекция** между $W_s(i)$ и множеството от начини за стигане до $i-s$ (обратното съответствие е „добави крачка s накрая“). Следователно $|W_s(i)| = dp[i-s]$ (по индукционна хипотеза, при $i-s \geq 0$; иначе $|W_s(i)| = 0$). Така:

$$|W(i)| = |W_1(i)| + |W_2(i)| + |W_3(i)| = dp[i-1] + dp[i-2] + dp[i-3] = dp[i]. \quad \checkmark$$

Пример

Нека $n = 4$.

i	0	1	2	3	4
$dp[i]$	1	1	2	4	7

Пресмятания:

- $dp[1] = dp[0] = 1$
- $dp[2] = dp[1] + dp[0] = 1 + 1 = 2$
- $dp[3] = dp[2] + dp[1] + dp[0] = 2 + 1 + 1 = 4$
- $dp[4] = dp[3] + dp[2] + dp[1] = 4 + 2 + 1 = 7$

Отговор: $dp[4] = 7$.

Проверка чрез изброяване (наредени суми на 4 от събираеми $\in \{1, 2, 3\}$):

$$1+1+1+1, \quad 2+1+1, \quad 1+2+1, \quad 1+1+2, \quad 2+2, \quad 3+1, \quad 1+3.$$

Общо 7 начина. ✓

Задача 4: Редукция $\text{Hamiltonian-Cycle} \leq_p \text{TSP}$ (NP-пълнота на TSP)

Условие: Да се докаже, че задачата TRAVELLING-SALESMAN (TSP) е **NP-пълна**, като се направи полиномиална редукция от задачата HAMILTONIAN-CYCLE (HC), за която приемаме за известно, че е NP-пълна.

Hamiltonian-Cycle (HC): Даден е ненасочен граф $G = (V, E)$. Съществува ли прост цикъл, който минава през всеки връх на G точно по веднъж (и се връща в началния връх)?

Travelling-Salesman (TSP), решаваща версия: Даден е пълен ненасочен граф върху n „града“ с неотрицателна теглова (разстояние) функция $d : \binom{V}{2} \rightarrow \mathbb{Q}_{\geq 0}$ и праг $B \in \mathbb{Q}_{\geq 0}$. Съществува ли *обиколка* (Хамилтонов цикъл в пълния граф), посещаваща всеки град точно по веднъж и връщаща се в началото, с обща дължина $\leq B$?

Какво трябва да докажем

За да е TSP **NP-пълна**, трябва да докажем две неща:

1. **TSP $\in$ NP** — има полиномиално проверим сертификат.
2. **TSP е NP-трудна** — съществува полиномиална редукция $A \leq_p \text{TSP}$ от някоя известна NP-пълна задача A . Тук избираме $A = \text{HC}$.

Стъпка 1: TSP $\in$ NP

Сертификат: наредба (пермутация) на градовете $\pi = (c_1, c_2, \dots, c_n)$, описваща предлаганата обиколка $c_1 \rightarrow c_2 \rightarrow \dots \rightarrow c_n \rightarrow c_1$.

Проверка за полиномиално време: (а) проверяваме, че π е пермутация на всичките n града (всеки се среща точно веднъж) — $O(n)$; (б) сумираме теглата на n -те ребра по обиколката, $\sum_{i=1}^n d(c_i, c_{i+1})$ (с $c_{n+1} = c_1$) — $O(n)$; (в) сравняваме сумата с B — $O(1)$. Общо $O(n)$, полиномиално. Следователно $\text{TSP} \in \text{NP}$. ✓

Стъпка 2: Конструкция на редукцията $f : \text{HC} \rightarrow \text{TSP}$

Нека $G = (V, E)$ е вход за HC с $n = |V|$ върха. Построяваме инстанция на TSP по следния начин:

- **Градове:** същите върхове V (затова n града).
- **Пълен граф с тегла:** за всяка двойка различни върхове $\{u, v\}$ задаваме

$$d(u, v) = \begin{cases} 1, & \text{ако } \{u, v\} \in E \quad (\text{реброто го има в } G), \\ 2, & \text{ако } \{u, v\} \notin E \quad (\text{липсващо ребро}). \end{cases}$$

- **Праг:** $B = n$.

Изходът на редукцията е $f(G) = (\text{пълнен граф върху } V, d, B = n)$.

Интуиция: „истинските“ ребра на G са евтини (тегло 1), а липсващите — скъпи (тегло 2). Обиколка използва точно n ребра, така че най-евтината възможна обиколка струва n само ако всичките ѝ ребра са истински — т.е. точно когато тя е Хамилтонов цикъл в G .

Стъпка 3: Редукцията е полиномиална

Пълната тегловна функция има $\binom{n}{2} = O(n^2)$ стойности; всяка се определя с една проверка „ $\{u, v\} \in E?$ “ (например чрез предварително изградена матрица на съседство за $O(n^2)$ или $O(n + m)$). Прагът $B = n$ се пресмята за $O(1)$. Общо построяването на $f(G)$ отнема $O(n^2)$ време — полиномиално спрямо размера на входа. ✓

Стъпка 4: Коректност на редукцията

Твърдение: G има Хамилтонов цикъл $\iff f(G)$ има обиколка с дължина $\leq n$.

Предварителна забележка: Всяка обиколка в TSP-инстанцията посещава всичките n града в цикъл, следователно използва **точно** n ребра.

($\Rightarrow$) **Ако G има Хамилтонов цикъл, то $f(G)$ е да-инстанция.**

Нека $C = v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_n \rightarrow v_1$ е Хамилтонов цикъл в G . Всяко негово ребро $\{v_i, v_{i+1}\}$ е в E , следователно има тегло $d(v_i, v_{i+1}) = 1$. Същата наредба е валидна TSP-обиколка с обща дължина

$$\sum_{i=1}^n d(v_i, v_{i+1}) = \underbrace{1 + 1 + \dots + 1}_n = n \leq B.$$

Значи $f(G)$ има обиколка с дължина $\leq n$. ✓

($\Leftarrow$) **Ако $f(G)$ е да-инстанция, то G има Хамилтонов цикъл.**

Нека T е обиколка в $f(G)$ с дължина $\leq n$. Тя използва точно n ребра, всяко с тегло ≥ 1 , така че

$$\text{дължина}(T) = \sum_{\text{ребро } e \in T} d(e) \geq \underbrace{1 + 1 + \dots + 1}_n = n.$$

Заедно с $\text{дължина}(T) \leq n$ получаваме $\text{дължина}(T) = n$ *точно*. Тъй като сборът от n числа, всяко ≥ 1 , е равен на n , всяко слагаемо е *точно* 1. Следователно **всяко** ребро на T има тегло 1, тоест принадлежи на E . Понеже T посещава всеки връх точно веднъж и се връща в началото, T е Хамилтонов цикъл, изцяло съставен от ребра на G — т.е. Хамилтонов цикъл в самия G . ✓

Двете посоки заедно дават еквивалентността, така че f е коректна редукция: $G \in \text{HC} \iff f(G) \in \text{TSP}$.

Заклучение

Покажахме, че $\text{TSP} \in \text{NP}$ (Стъпка 1) и че $\text{HC} \leq_p \text{TSP}$ чрез полиномиалната редукция f (Стъпки 2–4). Тъй като HC е NP-пълна (следователно NP-трудна), наличието на полиномиална редукция $\text{HC} \leq_p \text{TSP}$ прави и TSP NP-трудна. Заедно с $\text{TSP} \in \text{NP}$ това означава, че **TSP е NP-пълна.** ■

Пример

Нека G има върхове $V = \{1, 2, 3, 4\}$ и ребра

$$E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 1\}, \{1, 3\}\}.$$

G има Хамилтонов цикъл $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$. Редукцията дава пълен граф с праг $B = n = 4$ и матрица на разстоянията:

d	1	2	3	4
1	—	1	1	1
2	1	—	1	2
3	1	1	—	1
4	1	2	1	—

(единственото липсващо ребро е $\{2, 4\}$, затова $d(2, 4) = 2$; всички останали двойки имат тегло 1).

Посока ($\Rightarrow$): обиколката $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 1$ има дължина $1 + 1 + 1 + 1 = 4 \leq B$. Да-инстанция, както очаквахме.

Контрапример за интуицията на ($\Leftarrow$): нека сега G' е *пътят* $1 - 2 - 3 - 4$ (ребра $\{1, 2\}, \{2, 3\}, \{3, 4\}$), който **няма** Хамилтонов цикъл. Сега $d(4, 1) = d(1, 3) = d(2, 4) = 2$. Всяка обиколка е 4-цикъл и трябва да използва поне едно липсващо ребро:

$$1\ 2\ 3\ 4\ 1 : 1+1+1+2 = 5, \quad 1\ 2\ 4\ 3\ 1 : 1+2+1+2 = 6, \quad 1\ 3\ 2\ 4\ 1 : 2+1+2+2 = 7.$$

Минималната обиколка е с дължина $5 > 4 = B$ — няма обиколка $\leq B$. Това коректно отразява, че G' няма Хамилтонов цикъл. ✓

Обобщение

Задача	Подход / подзадача	Сложност
Лодки	сортиране + два указателя (алчно, exchange argument)	$O(n \log n)$
Bangaranga	разширен граф (състояние пеша/такси) + Dijkstra	$O((n + m) \log n)$
Стълба	$dp[i]$ — брой начини до стъпало i	$O(n)$
TSP е NP-пълна	редукция $HC \leq_p TSP$ (1/2 тегла, $B = n$)	$O(n^2)$ (редукция)

При **Лодки** ключът е алчният избор „сдвой най-тежкия с най-лекия, ако се събират“, чиято безопасност се доказва с exchange argument (всяко оптимално решение се пренарежда към този избор без влошаване). При **Bangaranga** ключът е да разпознаем скритото състояние и да го „вградим“ във върховете на нов граф — така цена, която зависи от контекст, става обикновено тегло на ребро, а задачата — стандартен най-къс път. При **Стълба** ключът е стандартният ДП-въпрос „кое е последното решение?“ — размерът на последната крачка — който води директно до рекурентната връзка $dp[i] = dp[i - 1] + dp[i - 2] + dp[i - 3]$. При **NP-пълнотата на TSP** ключът е да изберем тегла (1 за истинско ребро, 2 за липсващо) и праг $B = n$ така, че най-евтината обиколка да струва точно n *единствено когато* съществува Хамилтонов цикъл — и след това да докажем еквивалентността в двете посоки.